

Esercizio A1 – Complessità – Punti ≥ 8

- **Algoritmo A:**

$$T_A(n) = \begin{cases} \Theta(1) & \text{se } n \leq 1, \\ 5T_A(n/2) + cn & \text{se } n > 1. \end{cases}$$

Applichiamo il teorema delle ricorrenze lineari con partizione bilanciata: $a = 5, b = 2, \beta = 1$; si ottiene $\alpha = \log_2 5 > 2 > \beta$. Si applica quindi il caso 1 e la complessità è $\Theta(n^{\log_2 5} \approx \Theta(n^{2.32}))$.

- **Algoritmo B:**

$$T_B(n) = \begin{cases} \Theta(1) & \text{se } n \leq 1, \\ 3T_B(n-1) + c & \text{se } n > 1. \end{cases}$$

Applichiamo il teorema delle ricorrenze lineari di ordine costante: $a = 3, \beta = 0$. La complessità è quindi $\Theta(3^n)$.

- **Algoritmo C:**

$$T_C(n) = \begin{cases} \Theta(1) & \text{se } n \leq 1, \\ 9T_C(n/3) + cn^2 & \text{se } n > 1. \end{cases}$$

Applichiamo il teorema delle ricorrenze lineari con partizione bilanciata: $a = 16, b = 4, \beta = 2$; si ottiene $\alpha = \log_4 16 = 2 = \beta$. Si applica quindi il caso 2 e la complessità è $\Theta(n^2 \log n)$.

Fra i tre algoritmi, il terzo è preferibile perchè la sua complessità è inferiore.

Esercizio A2 – Tassografia – Punti ≥ 10

L'algoritmo si basa su una visita in profondità (DFS) con classificazione degli archi. Usiamo la procedura `hascycle()` già vista a lezione. Possono darsi due casi:

- **Il grafo contiene cicli:** In questo caso, classifichiamo G come *grafo generale non aciclico*.
- **Il grafo non contiene cicli:** Allora G è un DAG. Per stabilire se è anche un albero radicato orientato, calcoliamo l'in-degree di ogni nodo. Se tutti i nodi hanno in-degree 1 tranne un nodo con zero, allora è un albero radicato orientato. Altrimenti confermiamo che il grafo è un DAG e non è un albero.

Sia `hasCycle()` che valutare gli in-degree richiedono $O(m + n)$.

La procedura assume questa forma:

```

int categorizeGraph(GRAPH G)
int n = G.size()
int m = 0
if hasCycle(G) then
    | return 3                                % Grafo ciclico, quindi non-dag
else
    | int in = new int[1...n] = {0}                % In-degree
    | foreach u  $\in$  G.V() do
    | | foreach v  $\in$  G.adj(u) do
    | | | in[v] += 1
    | int count0 = 0
    | for u  $\in$  G.V() do
    | | if in[u] == 0 then
    | | | if count0 > 0 then
    | | | | return 2                                % C'è più di un radice, DAG
    | | | else if in[u] > 1 then
    | | | | return 2                                % Più di un arco entrante, DAG
    | return 1                                    % Albero radicato orientato

```

Esercizio A3 – Scendere nell'albero – Punti ≥ 12

Una soluzione banale consiste nel sondare tutti i nodi dell'albero e verificare, per ciascuno, se soddisfa la proprietà di minimo locale. Appena si trova un nodo che lo è, lo si restituisce. Questo approccio ha complessità $O(n)$, poiché nel caso pessimo bisogna visitare una porzione lineare dell'albero (in media almeno metà dei nodi).

Possiamo però ragionare in maniera più efficiente. Consideriamo la radice: se il suo valore è più piccolo di quello dei suoi figli, allora è già un minimo locale e l'algoritmo termina. Altrimenti, almeno uno dei figli ha valore minore della radice; scegliamo il figlio con valore più basso e scendiamo lungo quel ramo.

Se il nodo corrente è più piccolo dei suoi vicini, lo restituiamo come minimo locale; in caso contrario, uno dei suoi figli ha valore minore, e quindi possiamo proseguire verso quel figlio.

Iterando questo procedimento, si scende lungo un cammino radice–foglia: ad ogni passo si sceglie il figlio con valore più piccolo. Ci sono due possibilità:

- si incontra un nodo interno che è già un minimo locale, e l'algoritmo si arresta;
- si arriva a una foglia. In questo caso, la foglia è necessariamente un minimo locale: infatti il suo valore è minore di quello del genitore (da cui è stata scelta come figlio più piccolo), e non ha altri vicini.

Questo ragionamento garantisce che l'algoritmo trovi sempre un minimo locale lungo il cammino seguito, senza dover visitare l'intero albero.

Il codice è molto semplice:

```
TREE localMinimum(TREE T)
    if T.left == nil and T.right == nil then
        return T                                % Se siamo arrivati a una foglia, è un minimo locale
    else if T.val < T.left.val and T.val < T.right.val then
        return T                                % Le proprietà di minimo locale sono verificate in questo nodo
    else if T.left.val < T.val then
        return localMinimum(T.left)             % Il valore a sinistra è più piccolo, andiamo a sinistra
    else
        return localMinimum(T.right)           % Il valore a destra è più piccolo, andiamo a destra
```

Poiché stiamo operando su un albero perfetto, il percorso radice–foglia ha lunghezza $\Theta(\log n)$ e l'algoritmo ha complessità $O(\log n)$ (in quanto si può fermare prima).

Esercizio B1 – Mensa – Punti ≥ 6

Idea generale. Il problema si scompone in tre sottoproblemi indipendenti, uno per ciascuna categoria (primo, secondo, dessert). Per ogni categoria costruiamo un grafo di *flusso massimo* che assegna piatti agli studenti rispettando (i) le preferenze e (ii) le limitazioni di capacità (numero di porzioni disponibili). Il numero massimo di assegnazioni studente–piatto si ottiene sommando i tre massimi flussi di categoria.

Costruzione del grafo per una categoria fissata c . Sia $P_c = \{p \in P : t_p = c\}$ l'insieme dei piatti della categoria c . Costruiamo un grafo orientato $G_c = (V_c, E_c)$ con:

- un nodo sorgente s e un nodo pozzo t ;
- un nodo per ogni studente $s_i \in S$;
- un nodo per ogni piatto $p \in P_c$.

Archi e capacità:

- da s a ogni studente s_i : capacità 1 (ogni studente può ricevere al più un piatto della categoria c);
- da s_i a $p \in P_c$: capacità 1 se $p \in A_{s_i}$ (rispetto delle preferenze);
- da ogni piatto $p \in P_c$ a t : capacità m_p (numero di porzioni disponibili del piatto p).

Calcolo del flusso e soluzione complessiva. Eseguiamo un algoritmo di flusso massimo su G_c e otteniamo un valore F_c . Ripetiamo la procedura per $c = \text{primo}, \text{secondo}, \text{dessert}$.

Il numero massimo di assegnazioni studente–piatto è

$$F_{tot} = F_{primo} + F_{secondo} + F_{dessert}.$$